

Lab 8 Teoría de Computación

Este documento contiene las partes a y b de los problemas 1–3, y los problemas 4 y 5 completos. El código fuente también está en el repositorio (`src/`), y los datos crudos de profiling en `resultados/` (CSV) y resumidos en el `README.md`.

Metodología de medición (partes b). El tiempo se mide *dentro* de cada programa con `clock_gettime(CLOCK_MONOTONIC, &ts)` alrededor de la llamada a `function(n)`, así no se cuenta el arranque del proceso. Compilado con `gcc -O2`. En el problema 1 el contador es `volatile` para que el compilador no elimine ni colapse los ciclos. En los problemas 2 y 3 la salida de `printf` se redirige a `/dev/null` para medir el algoritmo y no la terminal. Corridas de menos de 1 s se repiten 5 veces y se reporta la mediana. El script de profiling valida que el número de operaciones contado por el programa coincida con la fórmula exacta derivada en cada parte a. Equipo de medición: VM Linux de 2 núcleos.

Notación: $\lfloor \cdot \rfloor$ y $\lceil \cdot \rceil$ son piso y techo; $n/2$ en C con enteros es $\lfloor n/2 \rfloor$.

Problema 1

Parte a

```
void function(int n) {
    int i, j, k, counter = 0;
    for (i = n/2; i <= n; i++) {
        for (j = 1; j+n/2 <= n; j++) {
            for (k = 1; k <= n; k = k*2) {
                counter++;
            }
        }
    }
}
```

Contamos cuántas veces se ejecuta la instrucción `counter++` (línea 6), que es la operación dominante. Los tres ciclos son **independientes** entre sí (ninguno depende de la variable de otro), así que el total es el producto de las iteraciones de cada uno.

Ciclo externo (i). i va de $\lfloor n/2 \rfloor$ a n con paso 1:

$$I_1 = n - \lfloor n/2 \rfloor + 1 = \lceil n/2 \rceil + 1.$$

Ciclo medio (j). La condición $j + \lfloor n/2 \rfloor \leq n$ equivale a $j \leq n - \lfloor n/2 \rfloor = \lceil n/2 \rceil$. Como j empieza en 1 con paso 1:

$$I_2 = \lceil n/2 \rceil.$$

Ciclo interno (k). k toma los valores $1, 2, 4, \dots, 2^m$ mientras $2^m \leq n$, o sea $m \leq \log_2 n$. El número de iteraciones es

$$I_3 = \lfloor \log_2 n \rfloor + 1.$$

Total.

$$T(n) = I_1 \cdot I_2 \cdot I_3 = (\lceil n/2 \rceil + 1) \lceil n/2 \rceil (\lfloor \log_2 n \rfloor + 1).$$

Para n grande, $\lceil n/2 \rceil + 1 \approx n/2$ y $\lceil n/2 \rceil \approx n/2$, por lo que

$$T(n) \approx \frac{n}{2} \cdot \frac{n}{2} \cdot \log_2 n = \frac{1}{4} n^2 \log_2 n.$$

Cota formal. Para $n \geq 2$: $\frac{n}{2} \leq \lceil n/2 \rceil \leq \lceil n/2 \rceil + 1 \leq n$ y $\log_2 n \leq \lfloor \log_2 n \rfloor + 1 \leq 2 \log_2 n$. Entonces

$$\frac{1}{4} n^2 \log_2 n \leq T(n) \leq 2 n^2 \log_2 n,$$

con lo que $T(n) = \Theta(n^2 \log n)$ y, en particular,

$$T(n) = O(n^2 \log n)$$

Verificación: para $n = 10$, $T = (5 + 1) \cdot 5 \cdot 4 = 120$, y para $n = 1000$, $T = 501 \cdot 500 \cdot 10 = 2\,505\,000$; ambos coinciden con el contador del programa en C.

Parte b — Implementación y profiling

Código completo (src/problema1.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* volatile evita que el compilador elimine o colapse los ciclos */
static volatile long long counter;

void function(int n) {
    int i, j, k;
    counter = 0;
    for (i = n / 2; i <= n; i++) {
        for (j = 1; j + n / 2 <= n; j++) {
            for (k = 1; k <= n; k = k * 2) {
                counter++;
            }
        }
    }
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Uso: %s <n>\n", argv[0]);
        return 1;
    }
    int n = atoi(argv[1]);
```

```

struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);
function(n);
clock_gettime(CLOCK_MONOTONIC, &t1);
double secs = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
printf("%d,%lld,%.9f\n", n, counter, secs);
return 0;
}

```

n	Operaciones	Tiempo	Tipo
1	2	0.12 μ s	medido
10	120	0.46 μ s	medido
100	17,850	43.61 μ s	medido
1,000	2,505,000	6.408 ms	medido
10,000	350,070,000	905.749 ms	medido
100,000	42,500,850,000	127.379 s	medido
1,000,000	5,000,010,000,000	14,985 s ($\approx$ 4.2 h)	estimado

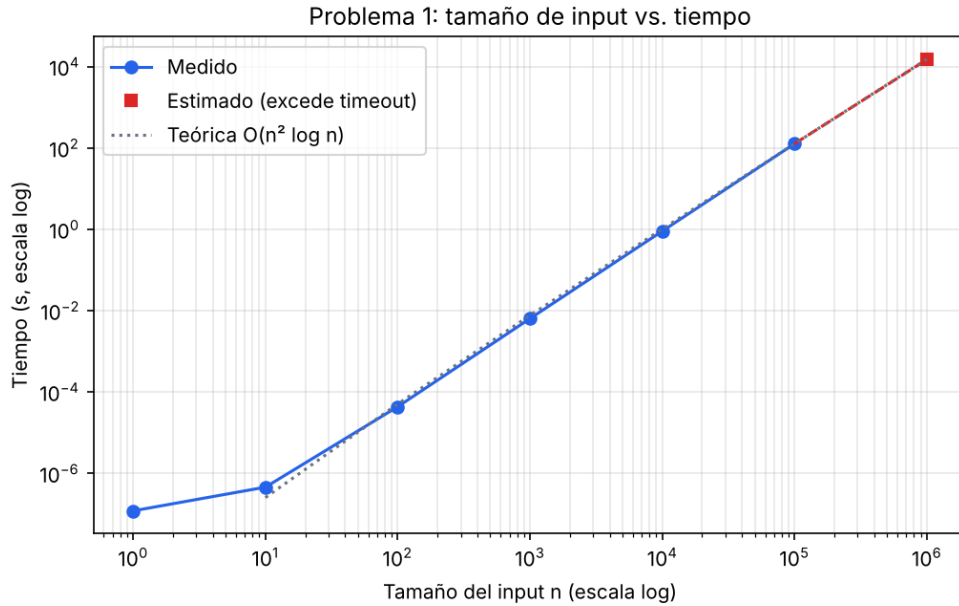


Figure 1: Problema 1: tamaño de input n vs. tiempo (escala log-log).

Con $n = 1\,000\,000$ el programa haría $\approx 5 \times 10^{12}$ incrementos (≈ 4 horas), por lo que ese punto se estimó con $t(n) \approx t(m) \cdot \text{ops}(n)/\text{ops}(m)$ a partir de la corrida medida más grande. Pasar de $n = 10^4$ a 10^5 multiplicó el tiempo por ≈ 141 , consistente con $10^2 \cdot \log(10^5)/\log(10^4) \approx 125$, lo que confirma el crecimiento $O(n^2 \log n)$.

Problema 2

Parte a

```
void function(int n) {
    if (n <= 1) return;
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("Sequence\n");
            break;
        }
    }
}
```

Caso base. Si $n \leq 1$ la función retorna inmediatamente: costo constante c_0 .

Ciclo externo. i va de 1 a n : n iteraciones.

Ciclo interno. Aunque su encabezado sugiere n iteraciones, en la *primera* iteración ($j = 1$) se ejecuta el `printf` y luego el `break`, que sale del ciclo interno. Por lo tanto el ciclo interno ejecuta **exactamente 1 iteración** por cada valor de i , con costo constante c_1 (inicializar j , comparar, imprimir, `break`).

Total.

$$T(n) = c_0 + \sum_{i=1}^n c_1 = c_0 + c_1 n.$$

El término dominante es lineal:

$$\boxed{T(n) = O(n)} \quad (\text{de hecho } \Theta(n))$$

El `printf` se ejecuta exactamente n veces para $n \geq 2$. El error típico es contar el ciclo interno como n iteraciones y concluir $O(n^2)$; el `break` lo impide.

Parte b — Implementación y profiling

Código completo (`src/problema2.c`):

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static long long impresiones = 0;

void function(int n) {
    if (n <= 1) return;
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("Sequence\n");
            impresiones++;
            break;
        }
    }
}
```

```

    }
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Uso: %s <n>\n", argv[0]);
        return 1;
    }
    int n = atoi(argv[1]);
    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);
    function(n);
    fflush(stdout);
    clock_gettime(CLOCK_MONOTONIC, &t1);
    double secs = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
    fprintf(stderr, "%d,%lld,%.9f\n", n, impresiones, secs);
    return 0;
}

```

n	Operaciones	Tiempo	Tipo
1	0	4.00 μ s	medido
10	10	18.24 μ s	medido
100	100	19.37 μ s	medido
1,000	1,000	29.52 μ s	medido
10,000	10,000	130.40 μ s	medido
100,000	100,000	1.209 ms	medido
1,000,000	1,000,000	11.760 ms	medido

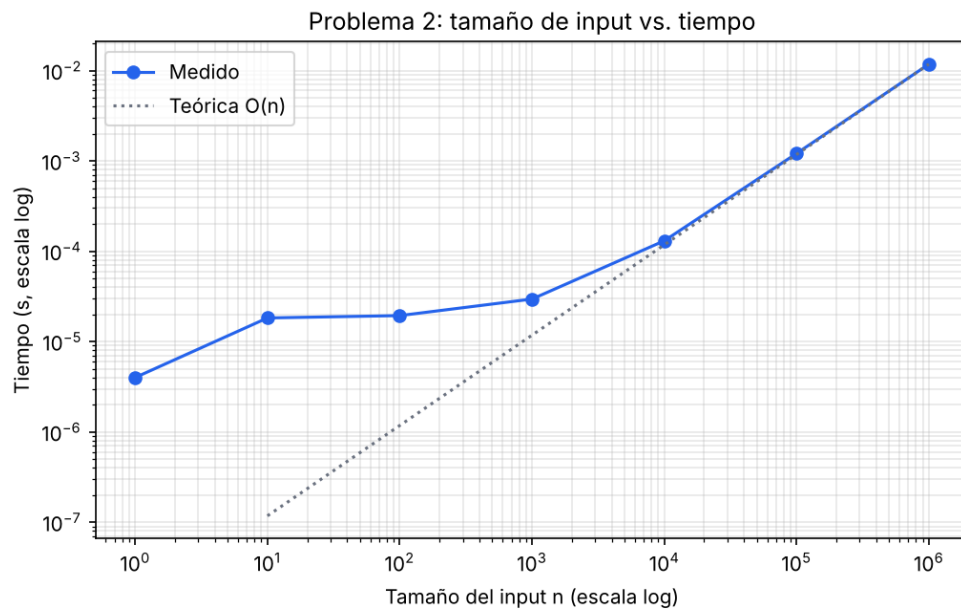


Figure 2: Problema 2: tamaño de input n vs. tiempo (escala log-log).

Por el **break**, el ciclo interno solo hace una iteración: el tiempo crece $\times 10$ por cada $\times 10$ en n a partir de $n \approx 10^4$. Para n pequeño domina el costo fijo de **printf/fflush**, lo que explica por qué los primeros puntos no siguen todavía la tendencia lineal.

Problema 3

Parte a

```
void function(int n) {
    int i, j;
    for (i = 1; i <= n/3; i++) {
        for (j = 1; j <= n; j += 4) {
            printf("Sequence\n");
        }
    }
}
```

Ciclo externo. i va de 1 a $\lfloor n/3 \rfloor$ con paso 1:

$$I_1 = \lfloor n/3 \rfloor.$$

Ciclo interno. j toma los valores $1, 5, 9, \dots, 1 + 4t$ mientras $1 + 4t \leq n$, es decir $t \leq (n-1)/4$. El número de iteraciones es

$$I_2 = \lfloor \frac{n-1}{4} \rfloor + 1 = \lceil n/4 \rceil.$$

El ciclo interno no depende de i , así que se repite igual en cada iteración externa.

Total.

$$T(n) = \sum_{i=1}^{\lfloor n/3 \rfloor} \lceil n/4 \rceil = \lfloor n/3 \rfloor \cdot \lceil n/4 \rceil \approx \frac{n}{3} \cdot \frac{n}{4} = \frac{n^2}{12}.$$

Cota formal. Para $n \geq 3$: $\frac{n}{6} \leq \lfloor n/3 \rfloor \leq \frac{n}{3}$ y $\frac{n}{4} \leq \lceil n/4 \rceil \leq \frac{n}{2}$, así que

$$\frac{1}{24} n^2 \leq T(n) \leq \frac{1}{6} n^2,$$

y por lo tanto

$$\boxed{T(n) = O(n^2)} \quad (\text{de hecho } \Theta(n^2))$$

Las constantes $1/3$ y $1/4$ (y el paso de 4) solo afectan la constante multiplicativa, no el orden de crecimiento.

Verificación: $n = 1000 \Rightarrow 333 \cdot 250 = 83\,250$ impresiones, igual que el programa.

Parte b — Implementación y profiling

Código completo (`src/problema3.c`):

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <time.h>

static long long impresiones = 0;

void function(int n) {
    int i, j;
    for (i = 1; i <= n / 3; i++) {
        for (j = 1; j <= n; j += 4) {
            printf("Sequence\n");
            impresiones++;
        }
    }
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Uso: %s <n>\n", argv[0]);
        return 1;
    }
    int n = atoi(argv[1]);
    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);
    function(n);
    fflush(stdout);
    clock_gettime(CLOCK_MONOTONIC, &t1);
    double secs = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
    fprintf(stderr, "%d,%lld,%.9f\n", n, impresiones, secs);
    return 0;
}

```

n	Operaciones	Tiempo	Tipo
1	0	4.32 μ s	medido
10	9	22.75 μ s	medido
100	825	32.12 μ s	medido
1,000	83,250	1.124 ms	medido
10,000	8,332,500	99.584 ms	medido
100,000	833,325,000	10.107 s	medido
1,000,000	83,333,250,000	1023.799 s	medido

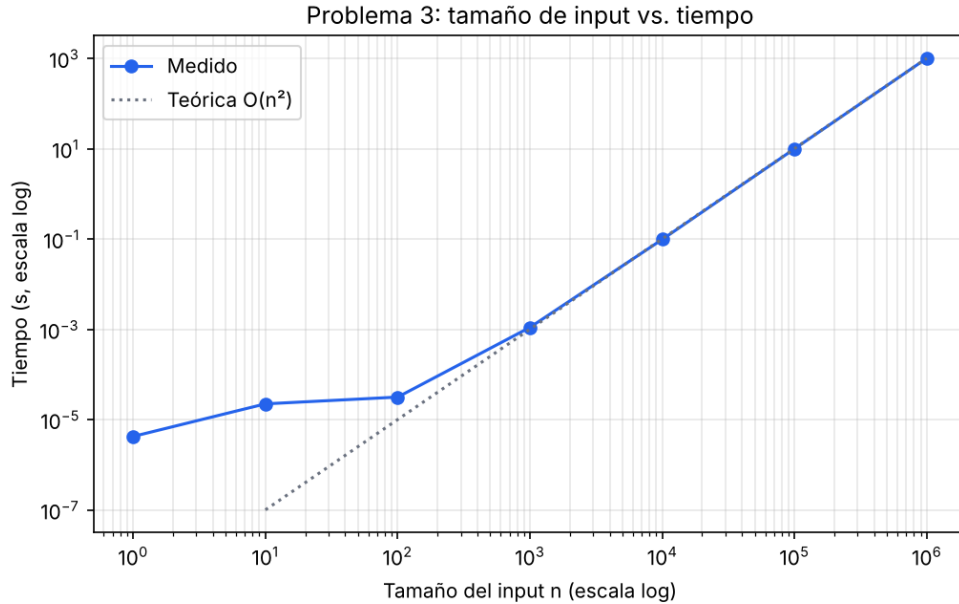


Figure 3: Problema 3: tamaño de input n vs. tiempo (escala log-log).

Cada $\times 10$ en n multiplica el tiempo por ≈ 100 , consistente con $O(n^2)$. Las operaciones son exactamente $\lfloor n/3 \rfloor \cdot \lceil n/4 \rceil \approx n^2/12$, como se verifica en la columna de operaciones.

Problema 4 — Mejor, promedio y peor caso

Medimos el costo en número de **comparaciones** contra elementos del arreglo, para un arreglo de tamaño n .

4.1 Búsqueda lineal

```
def linear_search(A, x):
    for i in range(len(A)):
        if A[i] == x:
            return i
    return -1
```

- **Mejor caso:** x está en la primera posición. Se hace 1 comparación: $T(n) = 1 = \Theta(1)$.
- **Peor caso:** x está en la última posición o no está. Se recorren los n elementos: $T(n) = n = \Theta(n)$.
- **Caso promedio:** supongamos que x está en el arreglo y que cada posición $i \in \{1, \dots, n\}$ es igualmente probable ($p = 1/n$). Encontrarlo en la posición i cuesta i comparaciones:

$$E[T] = \sum_{i=1}^n i \cdot \frac{1}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} = \Theta(n).$$

Si además x no está con probabilidad q , entonces $E[T] = (1 - q)\frac{n+1}{2} + qn$, que sigue siendo $\Theta(n)$ para cualquier q fijo.

4.2 Búsqueda binaria (arreglo ordenado)

```
def binary_search(A, x):
    lo, hi = 0, len(A) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if A[mid] == x: return mid
        elif A[mid] < x: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Cada iteración hace un número constante de comparaciones y reduce el intervalo a la mitad (o menos). Recurrencia: $T(n) = T(n/2) + c$.

- **Mejor caso:** x está justo en el primer mid. Una sola iteración: $T(n) = \Theta(1)$.
- **Peor caso:** x no está (o se encuentra en la última iteración). Desenrollando la recurrencia:

$$T(n) = T(n/2) + c = T(n/4) + 2c = \dots = T(n/2^k) + kc.$$

El intervalo llega a tamaño 1 cuando $n/2^k = 1 \Rightarrow k = \log_2 n$. Se hacen $\lfloor \log_2 n \rfloor + 1$ iteraciones: $T(n) = \Theta(\log n)$. (También sale del Teorema Maestro con $a = 1$, $b = 2$, $f(n) = \Theta(1) = \Theta(n^{\log_2 1})$, caso 2.)

- **Caso promedio:** veamos el árbol de decisiones de la búsqueda binaria. Tomemos $n = 2^k - 1$ por simplicidad. En el nivel d del árbol ($d = 1, \dots, k$) hay 2^{d-1} posiciones, y encontrar cualquiera de ellas cuesta d iteraciones. Si x está en una posición uniforme:

$$E[T] = \frac{1}{n} \sum_{d=1}^k d \cdot 2^{d-1} = \frac{(k-1)2^k + 1}{2^k - 1} = \frac{(k-1)(n+1) + 1}{n} \approx k - 1 \approx \log_2 n - 1.$$

(Se usó $\sum_{d=1}^k d 2^{d-1} = (k-1)2^k + 1$.) La mitad de los elementos están en el último nivel, así que el promedio queda a solo una iteración del peor caso: $E[T] = \Theta(\log n)$.

4.3 Quicksort

```
def quicksort(A, lo, hi):
    if lo < hi:
        p = partition(A, lo, hi) # n-1 comparaciones (Lomuto)
        quicksort(A, lo, p - 1)
        quicksort(A, p + 1, hi)
```

Particionar un subarreglo de tamaño n cuesta $n - 1$ comparaciones ($\Theta(n)$). Si el pivote queda en la posición q (con q elementos a su izquierda), la recurrencia general es

$$T(n) = T(q) + T(n - 1 - q) + (n - 1), \quad T(0) = T(1) = 0.$$

- **Mejor caso:** el pivote siempre divide el arreglo en dos mitades. $T(n) = 2T(n/2) + \Theta(n)$. Por el Teorema Maestro ($a = 2$, $b = 2$, $n^{\log_2 2} = n$, $f(n) = \Theta(n)$, caso 2):

$$T(n) = \Theta(n \log n).$$

Intuitivamente: el árbol de recursión tiene $\log_2 n$ niveles y cada nivel hace $\Theta(n)$ trabajo en total.

- **Peor caso:** el pivote es siempre el mínimo o el máximo (p.ej. arreglo ya ordenado con pivote en un extremo). Una de las partes queda vacía:

$$T(n) = T(n-1) + (n-1) = \sum_{m=1}^{n-1} m = \frac{n(n-1)}{2} = \Theta(n^2).$$

- **Caso promedio:** suponemos que todas las permutaciones son igualmente probables, así que el pivote cae en cualquier posición $q \in \{0, \dots, n-1\}$ con probabilidad $1/n$:

$$T(n) = (n-1) + \frac{1}{n} \sum_{q=0}^{n-1} [T(q) + T(n-1-q)] = (n-1) + \frac{2}{n} \sum_{q=0}^{n-1} T(q).$$

Multiplicamos por n y restamos la misma ecuación para $n-1$:

$$\begin{aligned} nT(n) &= n(n-1) + 2 \sum_{q=0}^{n-1} T(q) \\ (n-1)T(n-1) &= (n-1)(n-2) + 2 \sum_{q=0}^{n-2} T(q) \\ \Rightarrow nT(n) - (n-1)T(n-1) &= 2(n-1) + 2T(n-1) \\ \Rightarrow nT(n) &= (n+1)T(n-1) + 2(n-1). \end{aligned}$$

Dividiendo entre $n(n+1)$:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)}.$$

Desenrollando desde $T(1) = 0$ y usando $\frac{2(m-1)}{m(m+1)} = \frac{4}{m+1} - \frac{2}{m}$:

$$\frac{T(n)}{n+1} = \sum_{m=2}^n \left(\frac{4}{m+1} - \frac{2}{m} \right) = 2H_n - \frac{4n}{n+1},$$

donde $H_n = \sum_{m=1}^n 1/m \approx \ln n$. Entonces

$$T(n) = 2(n+1)H_n - 4n \approx 2n \ln n \approx 1.39 n \log_2 n = \Theta(n \log n).$$

El caso promedio está solo $\approx 39\%$ por encima del mejor caso.

Resumen

Algoritmo	Mejor caso	Caso promedio	Peor caso
Búsqueda lineal	$\Theta(1)$	$\Theta(n) \quad \left(\frac{n+1}{2}\right)$	$\Theta(n)$
Búsqueda binaria	$\Theta(1)$	$\Theta(\log n) \quad (\approx \log_2 n - 1)$	$\Theta(\log n)$
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n) \quad (\approx 1.39 n \log_2 n)$	$\Theta(n^2)$

Problema 5 — Verdadero o falso

Recordatorio de definiciones (para funciones positivas):

- $f = O(g) \iff \exists c > 0, n_0 : f(n) \leq c g(n) \forall n \geq n_0$.
- $f = \Omega(g) \iff \exists c > 0, n_0 : f(n) \geq c g(n) \forall n \geq n_0$.
- $f = \Theta(g) \iff \exists c_1, c_2 > 0, n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0$.

a) Si $f = \Theta(g)$ y $g = \Theta(h)$, entonces $h = \Theta(f)$. **VERDADERO**

Por hipótesis existen constantes positivas tales que, para $n \geq \max(n_0, n'_0)$:

$$c_1 g(n) \leq f(n) \leq c_2 g(n), \quad d_1 h(n) \leq g(n) \leq d_2 h(n).$$

Transitividad: sustituyendo las cotas de g en las de f :

$$c_1 d_1 h(n) \leq f(n) \leq c_2 d_2 h(n) \Rightarrow f = \Theta(h).$$

Simetría: despejando h de la desigualdad anterior:

$$\frac{1}{c_2 d_2} f(n) \leq h(n) \leq \frac{1}{c_1 d_1} f(n) \Rightarrow h = \Theta(f),$$

con constantes $\frac{1}{c_2 d_2}$ y $\frac{1}{c_1 d_1}$, ambas positivas. ■

b) Si $f = O(g)$ y $g = O(h)$, entonces $h = \Omega(f)$. **VERDADERO**

Por hipótesis, para n suficientemente grande: $f(n) \leq c g(n)$ y $g(n) \leq d h(n)$. **Transitividad de O :** $f(n) \leq c g(n) \leq c d h(n)$, así que $f = O(h)$.

Dualidad O/Ω : de $f(n) \leq c d h(n)$ se obtiene

$$h(n) \geq \frac{1}{c d} f(n) \quad \forall n \geq n_0,$$

que es exactamente la definición de $h = \Omega(f)$ con constante $\frac{1}{c d} > 0$. ■

c) $f(n) = \Theta(n^2)$ para el programa A(n). **FALSO:** $f(n) = \Theta(n^3)$

```
def A(n):
    atupla = tuple(range(0, n))
    S = set()
    for i in range(0, n):
        for j in range(i + 1, n):
            S.add(atupla[i:j])
```

La trampa es suponer que el cuerpo del ciclo es $O(1)$. Si lo fuera, el número de iteraciones sería $\sum_{i=0}^{n-1} (n-1-i) = \frac{n(n-1)}{2} = \Theta(n^2)$. Pero el cuerpo **no** es constante:

1. `atupla[i:j]` crea una tupla *nueva* copiando $j - i$ elementos: costo $\Theta(j - i)$. En Python, el slicing de tuplas copia (no es una vista).
2. `S.add(...)` debe calcular el hash de la tupla, y el hash de una tupla recorre todos sus elementos: costo $\Theta(j - i)$. Como todas las rebanadas son distintas (los elementos de `atupla` son distintos), en esperanza no hay comparaciones extra por colisiones, y la inserción en la tabla hash es $O(1)$ amortizado aparte del hash.
3. `tuple(range(0, n))` al inicio cuesta $\Theta(n)$, que es despreciable.

Por lo tanto el costo total es, sea $d = j - i$ la longitud de la rebanada:

$$f(n) = \Theta(n) + \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} \Theta(j - i).$$

Para cada longitud $d \in \{1, \dots, n - 1\}$, el número de pares (i, j) con $j - i = d$ y $0 \leq i < j \leq n - 1$ es $n - d$. Entonces

$$\sum_{d=1}^{n-1} d(n-d) = n \cdot \frac{(n-1)n}{2} - \frac{(n-1)n(2n-1)}{6} = \frac{n(n-1)[3n - (2n-1)]}{6} = \frac{(n-1)n(n+1)}{6} = \frac{n^3 - n}{6}.$$

Así, $f(n) = \Theta\left(\frac{n^3 - n}{6}\right) = \Theta(n^3)$. Como n^3 crece estrictamente más rápido que n^2 ($\lim n^3/n^2 = \infty$), no existe c_2 con $f(n) \leq c_2 n^2$; por lo tanto $f(n) \neq O(n^2)$ y el enunciado $f(n) = \Theta(n^2)$ es **falso**. (Lo que sí es cierto es $f(n) = \Omega(n^2)$.)

Verificación empírica (`src/problema5c.py`): si $f = \Theta(n^2)$, duplicar n multiplicaría el tiempo por ≈ 4 ; si $f = \Theta(n^3)$, por ≈ 8 .

n	tiempo (s)	$t(n)/t(n/2)$	t/n^2 (ns)	t/n^3 (ns)
100	0.0022	-	223.00	2.230
200	0.0235	10.52	586.52	2.933
400	0.1689	7.20	1055.66	2.639
800	1.3135	7.78	2052.39	2.565
1600	9.1230	6.95	3563.67	2.227

La razón al duplicar n se mantiene cerca de 8 y t/n^3 se mantiene aproximadamente constante, mientras que t/n^2 crece con n . Esto confirma $f(n) = \Theta(n^3)$.

Capturas de ejecución

A continuación, evidencia de que los programas de la parte b compilan y corren correctamente, con la salida real de terminal (formato `n,operaciones,tiempo_s`). Estas capturas se tomaron en una máquina local de verificación; la tabla completa de profiling oficial (hasta $n = 1\,000\,000$, medida con `clock_gettime` en una VM Linux) está en `README.md` y `resultados/` del repositorio, y resumida en la sección *Resultados* del `README`.

Problema 1

```
$ gcc -O2 -Wall -o problema1 src/problema1.c
$ ./problema1 1
1,2,0.000000000
$ ./problema1 10
10,120,0.000000000
$ ./problema1 100
100,17850,0.000000000
$ ./problema1 1000
1000,2505000,0.000000000
$ ./problema1 10000
10000,350070000,0.127000000
```

Problema 2

```
$ gcc -O2 -Wall -o problema2 src/problema2.c
$ ./problema2 1 > /dev/null
1,0,0.000000000
$ ./problema2 10 > /dev/null
10,10,0.000000000
$ ./problema2 100 > /dev/null
100,100,0.000000000
$ ./problema2 1000 > /dev/null
1000,1000,0.000000000
$ ./problema2 10000 > /dev/null
10000,10000,0.010000000
$ ./problema2 100000 > /dev/null
100000,100000,0.091000000
```

Problema 3

```
$ gcc -O2 -Wall -o problema3 src/problema3.c
$ ./problema3 1 > /dev/null
1,0,0.000000000
$ ./problema3 10 > /dev/null
10,9,0.000000000
$ ./problema3 100 > /dev/null
100,825,0.005000000
$ ./problema3 1000 > /dev/null
1000,83250,0.082000000
$ ./problema3 10000 > /dev/null
10000,8332500,7.771000000
```

Problema 5c — verificación empírica

```
$ python src/problema5c.py
n    tiempo (s)    t(n)/t(n/2)    t/n^2 (ns)    t/n^3 (ns)
```

100	0.0015	-	147.70	1.477
200	0.0121	8.21	303.08	1.515
400	0.0970	8.00	606.42	1.516
800	0.7625	7.86	1191.48	1.489
1600	8.6104	11.29	3363.43	2.102

En los cuatro casos el número de operaciones contado coincide con la fórmula exacta de la parte a correspondiente, y la razón $t(n)/t(n/2)$ de la última captura ronda 8, confirmando $\Theta(n^3)$ para el problema 5c.